



Apex Institute of Technology

Computer Science & Engineering

Experiment 10

Name: Shivam Abhepal

UID: 24BAI70214

Branch: B.E. CSE (AIML)

Section: 24AIT_KRG-G1

Semester: 4

Date of Performance: 24.04.2026

Subject Name: Database
Management System

Subject Code: 24CSH-298

AIM: This experiment focuses on designing and implementing a database trigger that automatically implements the functionality of a primary key, ensuring unique identification of records without manual intervention. It demonstrates how triggers can replicate the behavior of a stored procedure for enforcing data integrity.

OBJECTIVES:

- To create a database trigger that automatically enforces primary key constraints or generates unique key values, replicating the functionality of a stored procedure.

SOFTWARE REQUIREMENTS:

- Database Management System:
 - Oracle Database Express Edition (Oracle XE)
 - PostgreSQL Database
- Database Administration Tool / Client Tool:
 - Oracle SQL Developer (for Oracle XE)
 - pgAdmin (for PostgreSQL)

PROCEDURE/STEPS:

1. Identify the table requiring automated primary key enforcement.
2. Design a trigger that activates before insert operations.



Apex Institute of Technology

Computer Science & Engineering

3. Ensure that every new record receives a unique primary key automatically.
4. Validate the trigger by inserting multiple records and verifying unique keys.

PRACTICAL/EXPERIMENTAL STEPS:

Drop Table if Exists

DROP TABLE IF EXISTS customers;

Create Table

```
CREATE TABLE customers (  
id INT PRIMARY KEY,  
name VARCHAR(50),  
email VARCHAR(100)  
);
```

Create Sequence

```
CREATE SEQUENCE customers_seq  
START 1  
INCREMENT 1;
```

Create Trigger Function

```
CREATE OR REPLACE FUNCTION auto_generate_pk()  
RETURNS TRIGGER AS $$  
BEGIN  
IF NEW.id IS NULL THEN  
NEW.id := nextval('customers_seq');  
END IF;  
RETURN NEW;  
END;  
$$ LANGUAGE plpgsql;
```



Apex Institute of Technology

Computer Science & Engineering

Create Trigger

```
CREATE TRIGGER customers_trigger  
BEFORE INSERT ON customers  
FOR EACH ROW  
EXECUTE FUNCTION auto_generate_pk();
```

Insert Data

```
INSERT INTO customers (name, email) VALUES  
(‘Anhad’, ‘anhad@gmail.com’),  
(‘Rohit’, ‘rohit@gmail.com’),  
(‘Kartik’, ‘kartik@gmail.com’);
```

Display Output

```
SELECT * FROM customers;
```

OUTPUT:(Screenshots)

- Automatically generated unique IDs for each record.
- Records are inserted without manually specifying the primary key.

The screenshot displays a SQL IDE interface. The top pane shows the following SQL code:

```
8 CREATE SEQUENCE customers_seq  
9 START 1  
10 INCREMENT 1;  
11  
12 CREATE OR REPLACE FUNCTION auto_generate_pk()  
13 RETURNS TRIGGER AS $$  
14 BEGIN  
15 IF NEW.id IS NULL THEN  
16 NEW.id := nextval('customers_seq');  
17 END IF;  
18 RETURN NEW;  
19 END;  
20 $$ LANGUAGE plpgsql;  
21  
22 CREATE TRIGGER customers_trigger  
23 BEFORE INSERT ON customers  
24 FOR EACH ROW  
25 EXECUTE FUNCTION auto_generate_pk();  
26  
27 INSERT INTO customers (name, email) VALUES  
28 ('Anhad', 'anhad@gmail.com'),  
29 ('Rohit', 'rohit@gmail.com'),  
30 ('Kartik', 'kartik@gmail.com');  
31  
32 SELECT * FROM customers;
```

The bottom pane shows the 'Data Output' tab with the following table:

id	name	email
1	Anhad	anhad@gmail.com
2	Rohit	rohit@gmail.com
3	Kartik	kartik@gmail.com



Apex Institute of Technology

Computer Science & Engineering

```
Query Query History
8 CREATE SEQUENCE customers_seq
9 START 1
10 INCREMENT 1;
11
12 CREATE OR REPLACE FUNCTION auto_generate_pk()
13 RETURNS TRIGGER AS $$
14 BEGIN
15 IF NEW.id IS NULL THEN
16 NEW.id := nextval('customers_seq');
17 END IF;
18 RETURN NEW;
19 END;
20 $$ LANGUAGE plpgsql;
21
22 CREATE TRIGGER customers_trigger
23 BEFORE INSERT ON customers
24 FOR EACH ROW
25 EXECUTE FUNCTION auto_generate_pk();
26
27 INSERT INTO customers (name, email) VALUES
28 ('Anhad', 'anhad@gmail.com'),
29 ('Rohit', 'rohit@gmail.com'),
30 ('Kartik', 'kartik@gmail.com');
31
32 SELECT * FROM customers;
```

Data Output Messages Notifications

NOTICE: table "customers" does not exist, skipping
Successfully run. Total query runtime: 112 msec.
3 rows affected.

I/O ANALYSIS:

Input:

- Customer Name
- Email

Output:

- Automatically generated ID
- Customer Name
- Email



Apex Institute of Technology

Computer Science & Engineering

LEARNING OUTCOMES:

After completing this experiment, the learner will be able to:

- Understand the purpose and working of database triggers.
- Implement automated primary key functionality using triggers.
- Ensure data integrity without manual key assignment.
- Apply trigger-based automation in real-world enterprise applications such as Amazon, Flipkart, and Oracle Corporation.

RESULT:

The trigger was successfully created and executed. It automatically generated unique primary key values for each record during insertion, ensuring data integrity and eliminating manual errors.